

NEXT_PUBLIC_APP_NAME=FurMatch
NEXT_PUBLIC_APP_URL=http://localhost:3001

- NEXT_PUBLIC_API_URL=https://app-adopcion-backend-dev.azurewebsites.net
- NEXT_PUBLIC_API_URL=http://localhost:3000#  FurMatch Frontend - Documentación de Integración Backend

Última actualización: May 20, 2026

Resumen Ejecutivo

Este documento detalla cómo el frontend de **FurMatch** se conecta con el backend y base de datos. Contiene información sobre:

- Configuración de conexión y URLs
 - Autenticación (JWT)
 - Endpoints esperados por módulo funcional
 - Formato de requests/responses
 - Manejo de errores
 - Ejemplos de llamadas HTTP
-

Configuración de Conexión

URL Base del API

La URL del backend se configura via **variables de entorno públicas** (accesibles en el cliente):

```
NEXT_PUBLIC_API_URL = https://app-adopcion-backend-dev.azurewebsites.net
NEXT_PUBLIC_APP_NAME = FurMatch
NEXT_PUBLIC_APP_URL = http://localhost:3001 (frontend)
```

Nota: En desarrollo local, el frontend intenta usar `http://localhost:3000` por defecto si no está definida `NEXT_PUBLIC_API_URL`.

Cliente HTTP

El frontend usa **Axios** con configuración en [lib/http/api-client.js](#):

```
const apiClient = axios.create({
  baseURL: env.NEXT_PUBLIC_API_URL, // URL base dinámica
  timeout: 10_000, // 10 segundos de timeout
  headers: {
    "Content-Type": "application/json",
  },
});
```

```
},  
});
```

Autenticación

Token JWT

El frontend utiliza **JWT (JSON Web Tokens)** para autenticar peticiones:

1. **POST /api/auth/login** → El backend devuelve un **accessToken**
2. El frontend **guarda el token** en:
 - **localStorage** (clave: **furmatch.access_token**)
 - **Cookies** con **SameSite=Lax** y **max-age=86400** (1 día)
3. **Cada petición** incluye el header:

```
Authorization: Bearer <accessToken>
```

Interceptores de Axios

```
// Request interceptor: añade el token  
apiClient.interceptors.request.use((config) => {  
  const { accessToken } = getSessionTokens();  
  if (accessToken) {  
    config.headers.set("Authorization", `Bearer ${accessToken}`);  
  }  
  return config;  
});  
  
// Response interceptor: maneja expiración  
apiClient.interceptors.response.use(  
  (response) => response,  
  (error) => {  
    if (error.response?.status === 401 || error.response?.status === 403) {  
      clearSessionTokens();  
      // Redirige a login (excepto si el error es de /auth/login)  
      if (typeof window !== "undefined" &&  
!error.config?.url?.includes('/auth/')) {  
        window.location.href = "/login";  
      }  
    }  
    return Promise.reject(error);  
  }  
);
```

📍 Endpoints Esperados

☑ Autenticación (/api/auth)

Método	Endpoint	Request	Response	Status
POST	/api/auth/login	{ email, password }	{ token \ accessToken \ jwt }	200/401/403/500
POST	/api/auth/logout	—	—	200
POST	/api/auth/registro	Varia por rol (ver abajo)	Usuario creado	201/400/500
POST	/api/auth/recuperar-contrasena	{ email }	Confirmación	200/404/500
POST	/api/auth/reset-password	{ token, newPassword }	Confirmación	200/400/500

Nota: El endpoint de login es flexible con el nombre del token; acepta `token`, `accessToken`, `jwt`, o `data`.

👤 Perfil Adoptante (/api/adoptante)

Método	Endpoint	Descripción
POST	/api/adoptante/perfil	Crear perfil de adoptante
GET	/api/adoptante/perfil	Obtener perfil del adoptante autenticado
PUT	/api/adoptante/perfil	Actualizar perfil de adoptante

Campos esperados en Adoptante:

- Preferencias de etiquetas (tags)
- Información personal
- Localización
- Estado de la cuenta

🏠 Perfil Albergue (/api/albergue)

Método	Endpoint	Descripción
POST	/api/albergue/perfil	Crear perfil de albergue
GET	/api/albergue/perfil	Obtener perfil del albergue autenticado
PUT	/api/albergue/perfil	Actualizar perfil de albergue

Mascotas (/api/mascotas)

Método	Endpoint	Descripción	Parámetros
POST	/api/mascotas	Crear mascota	Body: payload con fotos/datos
GET	/api/mascotas/feed	Feed de mascotas para adoptantes	Query: filtros
GET	/api/mascotas/match	Mascotas compatibles	—
GET	/api/mascotas/mis-mascotas	Mascotas del albergue autenticado	Query: filtros
GET	/api/mascotas/:id	Detalle de una mascota	—
PUT	/api/mascotas/:id	Actualizar mascota	Body: datos a actualizar
PATCH	/api/mascotas/:id/estado	Cambiar estado (adoptada, etc.)	Body: { nuevoEstado }
DELETE	/api/mascotas/:id	Eliminar mascota	Body: { motivo }

Nota: El endpoint POST /api/mascotas tiene un **timeout de 60 segundos** (sube de fotos puede ser lenta).

Recomendaciones/Interacciones (/api/recomendaciones)

Método	Endpoint	Descripción
GET	/api/recomendaciones	Obtener recomendaciones pendientes
POST	/api/recomendaciones/:id/me-interesa	Marcar mascota como interesante
POST	/api/recomendaciones/:id/descartar	Descartar mascota
POST	/api/recomendaciones/deshacer	Deshacer última acción

Matches (/api/matches / /api/adopters/matches)

Método	Endpoint	Descripción	Parámetros
GET	/api/adopters/matches	Historial de matches del adoptante	Query: estado, fecha_desde, fecha_hasta, limit, offset
GET	/api/matches/:id	Detalle de un match específico	—

Estados esperados: pendiente, aceptado, rechazado, adoptado

Etiquetas/Tags (/api/etiquetas, /api/admin/etiquetas)

Método	Endpoint	Descripción	Roles
GET	/api/etiquetas	Listar etiquetas públicas	Todos
GET	/api/admin/etiquetas	Listar todas las etiquetas	Admin
POST	/api/admin/etiquetas	Crear etiqueta	Admin
PUT	/api/admin/etiquetas/:id	Actualizar etiqueta	Admin
DELETE	/api/admin/etiquetas/:id	Eliminar etiqueta	Admin
POST	/api/admin/etiquetas/:id/opciones	Añadir opción a etiqueta	Admin
DELETE	/api/admin/etiquetas/:id/opciones/:idOpcion	Eliminar opción	Admin

Formato de Tags:

```
{
  id,
  nombre,
  tipo,           // "select", "multiselect", "text", etc.
  opciones: [     // si tipo es select/multiselect
    { id, valor, nombre }
  ]
}
```

Usuarios Admin (/api/admin/usuarios)

Método	Endpoint	Descripción
GET	/api/admin/usuarios	Listar usuarios (con filtros: rol, estado)
PATCH	/api/admin/usuarios/:id/estado	Cambiar estado de usuario
DELETE	/api/admin/usuarios/:id	Eliminar usuario

Estados esperados: activo, bloqueado, pendiente_verificacion

Formato de Responses

El backend devuelve responses en **dos formatos posibles**, dependiendo del endpoint:

Formato 1: Wrapper (endpoints de detalle)

```
{
  "success": true,
  "data": { /* objeto principal */ }
}
```

El frontend extrae automáticamente `response.data` gracias al helper `extractData()` en los servicios.

Formato 2: Directo con metadata (endpoints de lista)

```
{
  "success": true,
  "data": [ /* array de objetos */ ],
  "meta": {
    "total": 50,
    "limit": 10,
    "offset": 0,
    "hasMore": true
  }
}
```

El frontend extrae `{ data: response.data, meta: response.meta }`.

Formato 3: Array directo (algunos endpoints de etiquetas)

```
[
  { id, nombre, tipo, opciones }
]
```

✖ Manejo de Errores

Códigos HTTP Esperados

Código	Significado	Acción del Frontend
200	OK	Procesa response
201	Created	Procesa response (create endpoints)
400	Bad Request	Muestra errores de validación
401	Unauthorized	Limpia tokens y redirige a login
403	Forbidden	Limpia tokens y redirige a login
413	Payload Too Large	Mensaje: "Fotos exceden tamaño máximo"

Código	Significado	Acción del Frontend
500	Server Error	Mensaje genérico de error

Formato de Error (esperado)

```
{
  "success": false,
  "message": "Descripción del error",
  "errors": [
    {
      "field": "email",
      "message": "Email inválido"
    }
  ]
}
```

El frontend parsea estos errores en funciones como `parseMascotaError()` en [features/albergue/services/mascota.service.js](#).

Ejemplos de Llamadas

Login

```
curl -X POST https://app-adopcion-backend-dev.azurewebsites.net/api/auth/login \
-H "Content-Type: application/json" \
-d '{
  "email": "adoptante@example.com",
  "password": "securePassword123"
}'

# Response esperado:
# 200 OK
# { "accessToken": "eyJhbGciOiJIUzI1NiIs..." }
# o { "token": "..." } o { "jwt": "..." }
```

Crear Mascota (con timeout de 60s)

```
curl -X POST https://app-adopcion-backend-dev.azurewebsites.net/api/mascotas \
-H "Authorization: Bearer <accessToken>" \
-H "Content-Type: application/json" \
-d '{
  "nombre": "Rex",
  "tipo": "perro",
  "descripcion": "perro de raza pura",
  "fecha": "2023-01-01",
  "estado": "disponible"
}'
```

```

    "raza": "Golden Retriever",
    "edad": 3,
    "tamaño": "grande",
    "descripcion": "Perro muy amigable",
    "etiquetas": [1, 2, 3],
    "fotos": [
      {
        "url": "base64-encoded-image-or-url",
        "tipo": "principal"
      }
    ]
  }'

```

Obtener Feed de Mascotas

```

curl -X GET 'https://app-adopcion-backend-
dev.azurewebsites.net/api/mascotas/feed?pagina=1&limite=10' \
-H "Authorization: Bearer <accessToken>"

# Response: { success: true, data: [mascota1, mascota2, ...], meta: {...} }

```

Registrar "Me Interesa"

```

curl -X POST https://app-adopcion-backend-
dev.azurewebsites.net/api/recomendaciones/123/me-interesa \
-H "Authorization: Bearer <accessToken>"

# Response: { success: true, data: { id, adoptante_id, mascota_id, estado, ...
} }

```

Información de Base de Datos

Módulos y Entidades Principales

1 Usuarios (Base)

- Tabla: **usuarios**
- Campos clave: **id**, **email**, **password** (hash), **rol**, **estado**, **created_at**, **updated_at**
- Roles: **adoptante**, **albergue**, **admin**

2 Adoptantes

- Tabla: **adoptantes** (extension de **usuarios**)

- Campos: `id`, `usuario_id` (FK), `nombre`, `telefono`, `ubicacion`, `preferencias_etiquetas` (JSON?), `estado_perfil`

3 Albergues

- Tabla: `albergues` (extension de `usuarios`)
- Campos: `id`, `usuario_id` (FK), `nombre_org`, `ubicacion`, `telefono`, `sitio_web`, `descripcion`, `logo_url`

4 Mascotas

- Tabla: `mascotas`
- Campos: `id`, `albergue_id` (FK), `nombre`, `tipo` (perro/gato/etc), `raza`, `edad`, `tamaño`, `descripcion`, `estado` (disponible/adoptada/bloqueada), `created_at`, `updated_at`

5 Fotos de Mascotas

- Tabla: `fotos_mascotas`
- Campos: `id`, `mascota_id` (FK), `url`, `tipo` (principal/adicional), `orden`, `created_at`

6 Etiquetas/Tags

- Tabla: `etiquetas`
- Campos: `id`, `nombre`, `tipo` (select/multiselect/text), `descripcion`, `activa`

7 Opciones de Etiquetas

- Tabla: `etiqueta_opciones`
- Campos: `id`, `etiqueta_id` (FK), `valor`, `nombre`, `orden`

8 Mascotas-Etiquetas (relación M-M)

- Tabla: `mascota_etiquetas`
- Campos: `mascota_id` (FK), `etiqueta_id` (FK), `valor_seleccionado` (puede ser JSON si multiselect)

9 Recomendaciones/Interacciones

- Tabla: `recomendaciones` (o `mascota_interacciones`)
- Campos: `id`, `adoptante_id` (FK), `mascota_id` (FK), `tipo` (me_interesa/descartada), `fecha`, `estado`

10 Matches

- Tabla: `matches`
- Campos: `id`, `adoptante_id` (FK), `mascota_id` (FK), `estado` (pendiente/aceptado/rechazado/adoptado), `fecha_match`, `fecha_finalizacion`, `notas`

Checklist para Implementación Backend

- ☐ Crear modelo de autenticación con JWT
- ☐ Implementar endpoints `/api/auth/*` (login, logout, registro, recuperar contraseña)
- ☐ Crear modelos de Adoptante, Albergue, Mascota
- ☐ Implementar CRUD de mascotas con validación de fotos (max 413 si excede tamaño)
- ☐ Crear tabla de Etiquetas y relación M-M con Mascotas
- ☐ Implementar endpoints de Recomendaciones (me_interesa, descartar, deshacer)
- ☐ Crear sistema de Matches (puede ser lógica en backend o trigger en BD)
- ☐ Implementar endpoints de Admin (usuarios, etiquetas)
- ☐ Middleware de autenticación para proteger endpoints privados
- ☐ Middleware para validar JWT (Bearer token)
- ☐ Manejo de CORS (frontend está en `http://localhost:3001`)
- ☐ Rate limiting o lockout después de 3 intentos fallidos de login
- ☐ Logger/auditoría de cambios en base de datos

Estructura de Carpetas del Frontend

```
frontend/
├── app/                                # Next.js app router
│   ├── (auth)/                        # Rutas de autenticación
│   │   ├── login/
│   │   ├── registro/
│   │   ├── recuperar-contrasena/
│   │   └── reset-password/
│   ├── adoptante/                    # Portal de adoptante
│   │   ├── feed/
│   │   ├── matches/
│   │   └── perfil/
│   ├── albergue/                     # Portal de albergue
│   │   ├── mascotas/
│   │   ├── publicar/
│   │   └── perfil/
│   └── admin/                         # Panel de administración
│       ├── dashboard/
│       ├── usuarios/
│       └── tags/
├── features/                          # Lógica por módulo
│   ├── auth/                         # Esquemas, componentes, servicios de auth
│   ├── adoptante/                    # Servicios y componentes del adoptante
│   ├── albergue/                     # Servicios y componentes del albergue
│   ├── admin/                        # Panel admin
│   └── shared/                       # Componentes compartidos
├── lib/
│   ├── auth/                         # Token storage, auth service
│   ├── http/                         # API client (axios con interceptores)
│   └── utils/                        # Utilidades
```

Referencias

- **Cliente HTTP:** [lib/http/api-client.js](#)
- **Token Storage:** [lib/auth/token-storage.js](#)
- **Servicios Adoptante:** [features/adoptante/services/](#)
- **Servicios Albergue:** [features/albergue/services/](#)
- **Servicios Admin:** [features/admin/services/](#)
- **Configuración:** [config/env.js](#)

Contacto & Notas

- **Rama Actual:** **deploy** (sincronizada con **develop** el 2026-05-20)
- **URL Frontend (Dev):** <http://localhost:3001>
- **URL Backend (Configurado):** <https://app-adopcion-backend-dev.azurewebsites.net>
- **Timeout de Requests:** 10 segundos (60 segundos para POST [/api/mascotas](#))

Última sincronización: Merge de **develop** a **deploy** realizado el 2026-05-20. Backup creado: **deploy-backup-20260520-140052**.

Generado automáticamente. Actualizar cuando haya cambios significativos en los endpoints o autenticación.